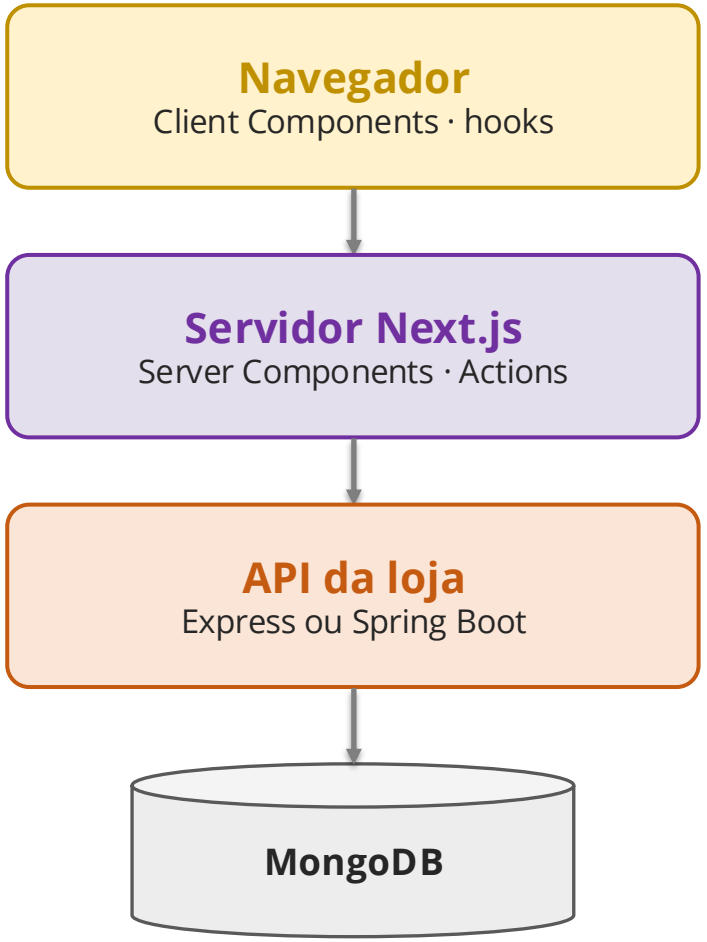
PUC Minas

Desenvolvimento de Aplicações com React e Next.js

Prof. Rommel Vieira Carneiro

Tópicos

- **Da página gerada no servidor à interface reativa**
  - Por que React, a loja-web e o contrato da API
- **Fundamentos do React**
  - JSX, props, composição, condicionais, listas, eventos e estado
- **Efeitos e hooks**
  - useEffect, consumo de API, hooks personalizados, useRef, Context e useReducer
- **Next.js e o App Router**
  - Rotas, layouts, rotas dinâmicas, query string e navegação
- **Estratégias de renderização**
  - CSR, SSR, SSG, Server e Client Components, Suspense e streaming
- **Busca de dados e atualizações**
  - Cache e revalidação, Server Actions e Route Handlers



Bibliografia da Unidade

BANKS, A.; PORCELLO, E. Learning React. 2. ed. O'Reilly, 2020.
Bibliografia básica – componentes, hooks e o modelo mental do React.

REACT. React Documentation (react.dev)
Documentação oficial – "Learn" e "Reference", atualizada para o React 19.

VERCEL. Next.js Documentation (nextjs.org/docs)
Documentação oficial – App Router, renderização, busca de dados e cache.

MDN. Fetch API e CORS (developer.mozilla.org)
Referência – requisições HTTP no navegador e compartilhamento entre origens.

PUC Minas

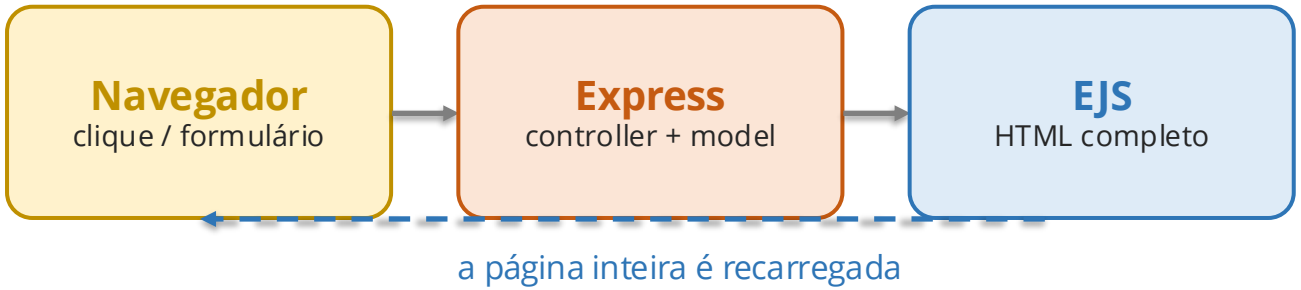
React e Next.js

Da Página Gerada no Servidor à Interface Reativa

Prof. Rommel Vieira Carneiro

Relembrando: o MVC com EJS

Na **loja-mvc**, o servidor monta o HTML completo a cada requisição. Qualquer mudança na tela – filtrar, paginar, adicionar ao carrinho – exige **uma nova requisição e uma nova página**.



- **Ótimo** para conteúdo, SEO e simplicidade.
- **Limitado** para interfaces muito interativas: busca enquanto digita, carrinho, validação instantânea.
- JavaScript "solto" para manipular o DOM cresce rápido e fica difícil de manter.

DOM "na mão"

```
// Imperativo: você diz COMO mudar a tela
const span = document.querySelector('#contador');
botao.addEventListener('click', () => {
  qtd++;
  span.textContent = qtd;   // e se outro trecho também mudar?
});
```

A ideia do React
Declarar **como a tela deve ficar para cada estado**.
O React calcula e aplica as mudanças no DOM.

O que é o React

React é uma biblioteca JavaScript para construir interfaces a partir de **componentes**: funções que recebem dados e devolvem a descrição de um pedaço da tela.

UI = f(estado)

Componentes

Peças reutilizáveis e combináveis: Preco, CartaoProduto, GradeProdutos...

Declarativo

Você descreve o resultado; o React atualiza o DOM de forma eficiente.

Estado reativo

Quando o estado muda, o componente é renderizado de novo.

Ecossistema

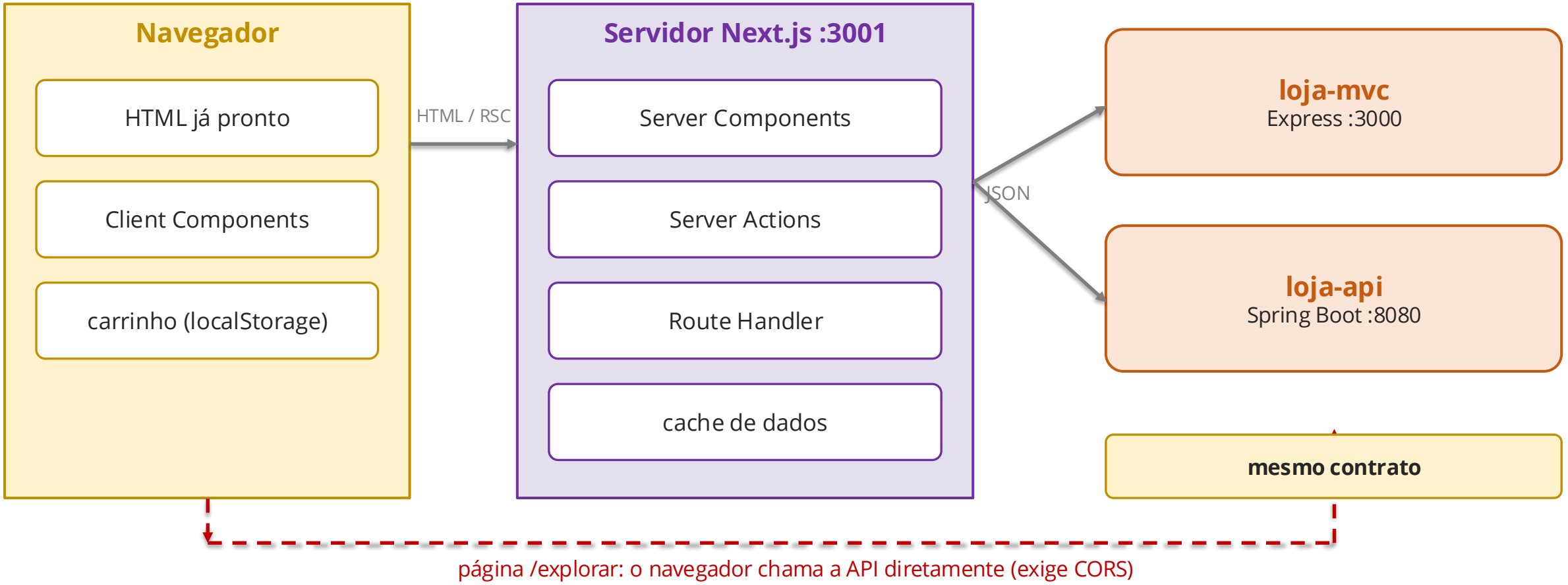
Next.js, React Native, bibliotecas de UI, testes e ferramentas.

React é biblioteca; Next.js é framework

Responsabilidade	React (sozinho)	Next.js (sobre o React)
Componentes e estado	✓	✓ (é o próprio React)
Rotas e navegação	biblioteca extra	✓ baseado em pastas (App Router)
Renderização no servidor	exige configuração	✓ padrão: Server Components
Busca de dados e cache	por sua conta	✓ fetch com cache e revalidação
Formulários e escrita	por sua conta	✓ Server Actions
Build e otimizações	Vite ou similar	✓ integrado (imagens, fontes, divisão de código)

Por que Next.js na disciplina?
A própria documentação do React recomenda começar com um framework. O Next.js também retoma temas da disciplina: servidor, HTTP, cache e integração com APIs.

A aplicação da unidade: loja-web



O contrato único da API

As duas APIs (Express e Spring) implementam **o mesmo contrato**. O front-end escolhe o back-end só pela variável de ambiente `API_URL` – nenhuma linha de código muda.

Método e caminho	Descrição	Respostas
GET /api/produtos?nome&categoria&page&size	Lista paginada, ordenada por nome	200
GET /api/produtos/{id}	Detalhe de um produto	200 · 404
POST /api/produtos	Cria um produto	201 + Location · 400 · 404
PUT /api/produtos/{id}	Atualiza um produto	200 · 400 · 404
DELETE /api/produtos/{id}	Remove um produto	204 · 404
GET /api/categorias	Lista as categorias	200

GET /api/produtos

```
{ "content": [ { "id": "…", "nome": "…", "preco": 89.9, … } ],
  "page": { "size": 8, "number": 0, "totalElements": 14, "totalPages": 2 } }
```

Erros no formato **Problem Details** (RFC 9457): `{ title, status, detail, erros }`. O arquivo **contrato-api.md** (pasta Fontes) descreve tudo.

Criando o projeto

terminal

```
# Cria o projeto sem perguntas (opções da disciplina)
$ npx create-next-app@latest loja-web \
    --js --eslint --app --use-npm \
    --no-tailwind --no-src-dir --no-react-compiler

$ cd loja-web
$ npm install server-only

# package.json – porta 3001 (a 3000 é da API Express)
#   "dev":   "next dev --port 3001",
#   "start": "next start --port 3001",

$ npm run dev
  ▲ Next.js 16.3.6
  - Local:  http://localhost:3001
```

Escolhas da disciplina

- **JavaScript**: o foco é o React; TypeScript fica como evolução natural.
- **Sem Tailwind**: um único `globals.css`, para não misturar dois assuntos.
- **App Router**: o modelo atual do Next.js (o antigo Pages Router é legado).
- **Porta 3001** nos scripts `dev` e `start`: a 3000 é da API Express.
- Sem as opções, o assistente pergunta cada item; o padrão dele é TypeScript + Tailwind.

Estrutura do projeto

```
loja-web/
├── app/                    rotas (App Router)
│   ├── layout.js           layout raiz
│   ├── page.js             página inicial  →  /
│   ├── produtos/           →  /produtos, /produtos/[id]…
│   ├── explorar/           →  /explorar
│   ├── carrinho/           →  /carrinho
│   └── api/sugestoes/      →  Route Handler
├── components/             componentes reutilizáveis
├── contexts/               Context API (carrinho)
├── hooks/                  hooks personalizados
├── lib/
│   ├── api.js              acesso à API (só servidor)
│   └── actions.js          Server Actions
├── .env.local              API_URL (não versionar)
├── next.config.mjs
└── package.json
```

- Só a pasta **app/** define rotas; as demais são convenções da equipe.
- O alias **@/** aponta para a raiz: `import Preco from '@/components/Preco'`.
- A separação lembra as camadas já vistas: **lib/** fala com a API; **components/** é a apresentação.
- O **.env.local** guarda configuração local e fica fora do Git.

PUC Minas

React e Next.js

Fundamentos do React

Prof. Rommel Vieira Carneiro

Componentes: funções que devolvem UI

components/Preco.js

```
// Componente de apresentação: formata um valor como moeda brasileira
const formato = new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' });

export default function Preco({ valor }) {
  return <span className="preco">{formato.format(valor)}</span>;
}
```

uso

```
<Preco valor={89.9} />            // → <span class="preco">R$ 89,90</span>
<Preco valor={produto.preco} />
```

Criar o formatador fora da função evita recriá-lo a cada renderização.

- Um componente é uma **função JavaScript** que devolve **JSX**.
- O nome começa com **letra maiúscula** – é assim que o React diferencia `<Preco>` de `<span>`.
- Um arquivo por componente, exportado com `export default`.
- Componentes **puros**: mesmas entradas, mesma saída – sem efeitos colaterais durante a renderização.

JSX: HTML dentro do JavaScript?

JSX é uma extensão de sintaxe que o compilador transforma em chamadas de função. Parece HTML, mas segue regras do JavaScript:

HTML

```
<!-- HTML / EJS -->
<article class="cartao">
  <label for="nome">Nome</label>
  <input id="nome">
  <img src="foto.png">
  <p><%= produto.nome %></p>
</article>
```

JSX

```
// JSX
<article className="cartao">
  <label htmlFor="nome">Nome</label>
  <input id="nome" />
  <img src="foto.png" alt="" />
  <p>{produto.nome}</p>
</article>
```

class → className
for → htmlFor

Toda tag é fechada
<input />, <img />

{ } insere expressões
{produto.nome}, {total * 2}

Um único elemento raiz
ou um fragmento <>...</>

Props: passando dados para componentes

components/CartaoProduto.js

```
import Link from 'next/link';
import Preco from './Preco';

// Recebe os dados por props e compõe outros componentes
export default function CartaoProduto({ produto }) {
  return (
    <article className="cartao">
      <span className="etiqueta">{produto.categoria}</span>
      <h3>
        <Link href={`/produtos/${produto.id}`}>{produto.nome}</Link>
      </h3>
      <Preco valor={produto.preco} />
      {produto.disponivel ? (
        <p className="ok">{produto.estoque} em estoque</p>
      ) : (
        <p className="esgotado">Esgotado</p>
      )}
    </article>
  );
}
```

- **Props** são os “parâmetros” do componente: chegam como **um objeto**, normalmente desestruturado.
- São **somente leitura**: o componente nunca altera as próprias props.
- Qualquer valor JavaScript: texto, número, objeto, função, JSX...

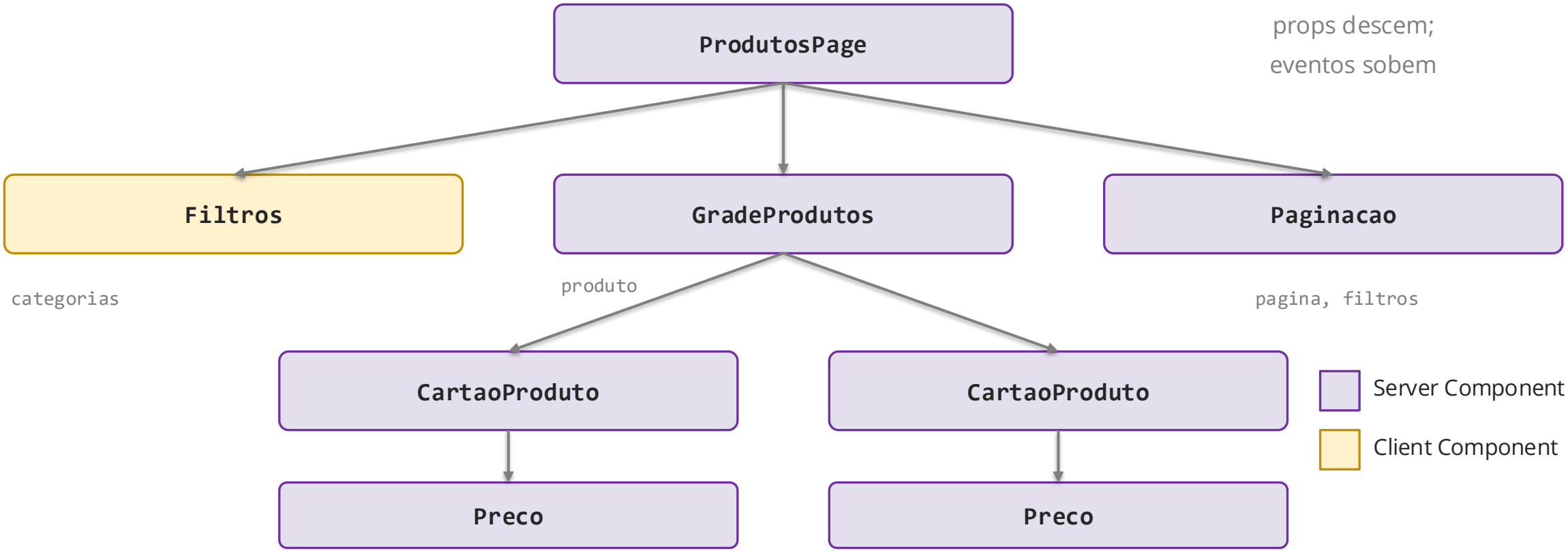
uso

```
// quem usa decide os dados
<CartaoProduto produto={p} />

// dentro: ({ produto }) => …
```

Composição: interfaces como árvores

Telas são montadas **combinando componentes pequenos**. Os dados descem pela árvore via props.



A prop especial children

components/NavLink.js

```
import Link from 'next/link';
import { usePathname } from 'next/navigation';

// Link que destaca a seção atual – precisa do navegador (usePathname)
export default function NavLink({ href, children }) {
  const pathname = usePathname();
  const ativo = href === '/' ? pathname === '/' : pathname.startsWith(href);
  return (
    <Link href={href} className={ativo ? 'ativo' : undefined}>
      {children}
    </Link>
  );
}
```

uso em Cabecalho.js

```
<NavLink href="/produtos">Produtos</NavLink>
<NavLink href="/explorar">Explorar (CSR)</NavLink>
```

- O que fica **entre as tags** de um componente chega na prop **children**.
- Permite criar “moldes” reutilizáveis: `NavLink`, `Card`, `Modal`, `Layout`...
- `NavLink` **encapsula** o `Link` do Next.js e acrescenta um comportamento: destacar a seção atual.
- O **layout** do Next.js usa exatamente esta ideia: `children` é a página da rota atual.

Renderização condicional

retorno antecipado

```
import CartaoProduto from './CartaoProduto';

export default function GradeProdutos({ produtos }) {
  if (produtos.length === 0) {
    return <p className="vazio">Nenhum produto encontrado.</p>;
  }
```

operador ternário

```
{produto.disponivel ? (
  <p className="ok">{produto.estoque} em estoque</p>
) : (
  <p className="esgotado">Esgotado</p>
)}
```

operador &&

```
{produto.descricao && <p>{produto.descricao}</p>}
```

Forma	Quando usar
if + return	Casos à parte (vazio, erro)
cond ? A : B	Um ou outro
cond && A	Mostrar ou não mostrar

Cuidado com o &&
`{lista.length && <Grade/>}` mostra **0** na tela quando a lista está vazia. Use `lista.length > 0 &&`.

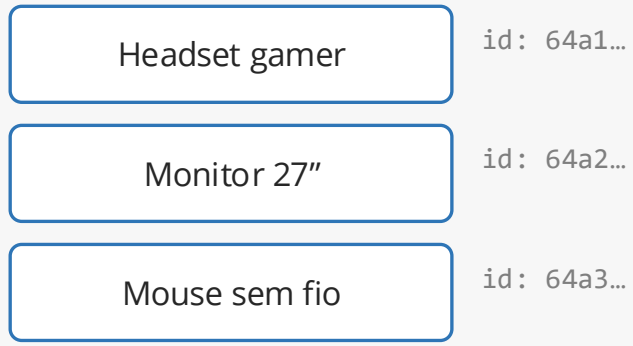
Listas e keys

components/GradeProdutos.js

```
return (
  <div className="grade">
    {produtos.map((produto) => (
      <CartaoProduto key={produto.id} produto={produto} />
    ))}
  </div>
);
```

- Listas são geradas com **map**: cada item vira um elemento JSX.
- Cada item precisa de uma **key** única e estável entre os irmãos.
- Use o **id** do dado. O índice do array só serve para listas que nunca mudam de ordem.

Por que a key importa?



- A key identifica **qual item é qual** entre duas renderizações.
- Sem ela (ou com índices), ao filtrar ou reordenar o React pode reaproveitar o elemento errado – e levar junto estado e foco.

Eventos

components/BotaoAdicionar.js

```
import { useState } from 'react';
import { useCarrinho } from '@/contexts/CarrinhoContext';

export default function BotaoAdicionar({ produto }) {
  const { dispatch } = useCarrinho();
  const [adicionado, setAdicionado] = useState(false);

  function adicionar() {
    dispatch({ tipo: 'adicionar', produto });
    setAdicionado(true);
  }

  return (
    <button onClick={adicionar} disabled={!produto.disponivel}>
      {!produto.disponivel ? 'Indisponível'
        : adicionado ? 'Adicionado ✓' : 'Adicionar ao carrinho'}
    </button>
  );
}
```

- Eventos em **camelCase**: `onClick`, `onChange`, `onSubmit`.
- Passe a **função**, não o resultado:

✓ `onClick={adicionar}`

✗ `onClick={adicionar()}` – executa ao renderizar

- Com argumentos, use uma arrow function: `onClick={() => remover(id)}`.

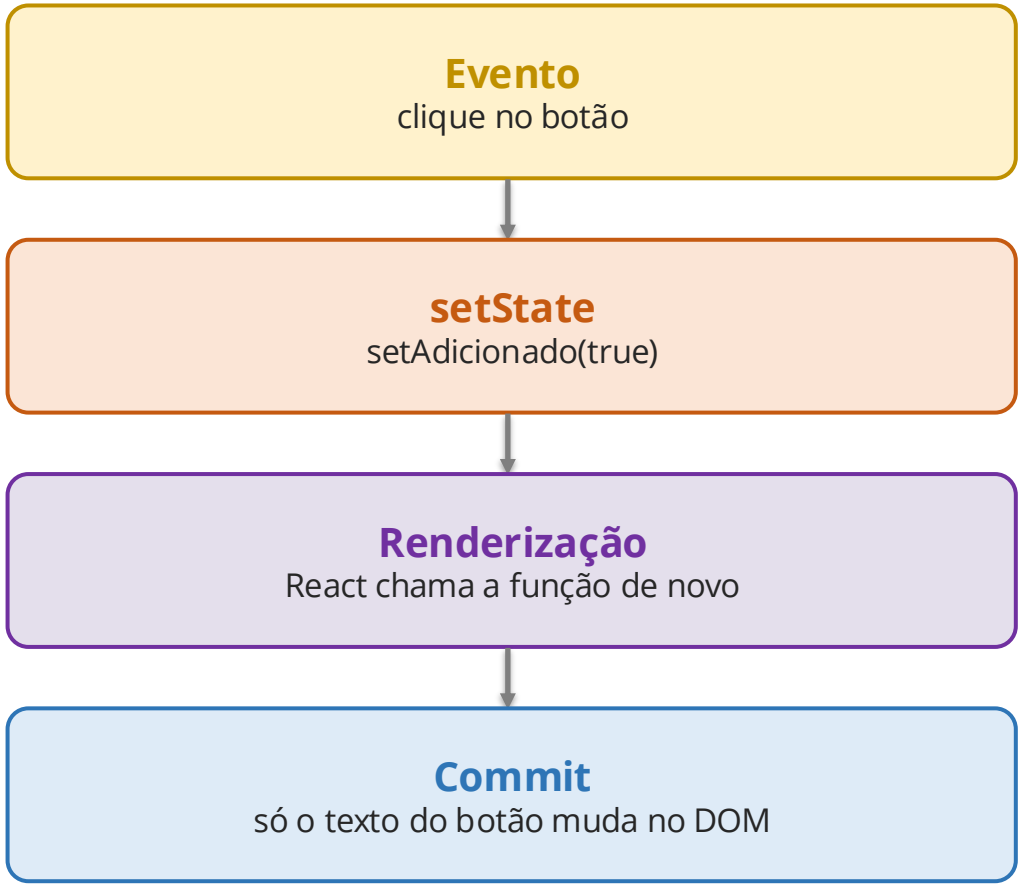
components/Filtros.js

```
function aplicar(evento) {
  evento.preventDefault();
  const dados = new FormData(evento.currentTarget);
```

Estado com useState

```
const [adicionado, setAdicionado] = useState(false);
//     valor atual   função que altera  valor inicial
```

- **Estado** é a memória do componente entre renderizações.
- Chamar `setAdicionado(true)` **agenda** uma nova renderização com o novo valor.
- Uma variável comum (`let adicionado`) não funciona: some a cada renderização e não avisa o React.
- Cada instância do componente tem **seu próprio** estado.



Props × estado

	Props	Estado
Quem define	O componente pai	O próprio componente
Pode mudar?	Não – somente leitura	Sim – via função set
Mudança provoca	Nova renderização do pai → novas props	Nova renderização do componente
Exemplo na loja	produto em CartaoProduto	adicionado em BotaoAdicionar

- **Imutabilidade**: nunca altere o estado diretamente – crie um novo valor.

✗ `itens.push(novo)` ✓ `setItens([...itens, novo])`

- **Estado derivado**: o que pode ser calculado a partir de props/estado **não** vira estado. Ex.: `total` do carrinho é calculado, não guardado.

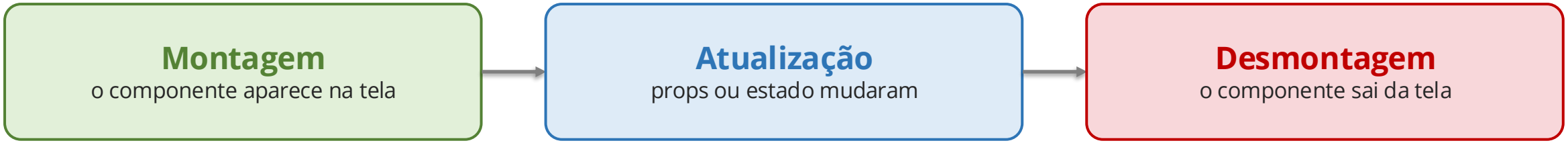
PUC Minas

React e Next.js

Efeitos e Hooks

Prof. Rommel Vieira Carneiro

Ciclo de vida de um componente



Momento	Classes (legado)	Componentes de função (atual)
Montagem	componentDidMount	useEffect(() => { … }, [])
Atualização	componentDidUpdate	useEffect(() => { … }, [dep])
Desmontagem	componentWillUnmount	função de limpeza devolvida pelo efeito

Mude a pergunta
Em vez de "o que fazer em cada fase?", pergunte: **"com o que este componente precisa ficar sincronizado?"** (um timer, uma requisição, o localStorage). Isso é um **efeito**.

useEffect: sincronização externa

```
useEffect(() => {
  // 1. efeito: executa DEPOIS da renderização
  return () => {
    // 2. limpeza: antes do próximo efeito e ao desmontar
  };
}, [dependencias]);   // 3. quando executar de novo
```

Dependências	O efeito executa...
[]	uma vez, após a montagem
[a, b]	na montagem e quando a ou b mudar
(omitido)	após toda renderização – raramente desejado

hooks/useDebounce.js

```
// Hook personalizado: devolve o valor só depois que ele "para de mudar"
export function useDebounce(valor, atraso = 400) {
  const [valorAtrasado, setValorAtrasado] = useState(valor);

  useEffect(() => {
    const timer = setTimeout(() => setValorAtrasado(valor), atraso);
    return () => clearTimeout(timer);  // cancela se o valor mudar antes
  }, [valor, atraso]);

  return valorAtrasado;
}
```

- A **limpeza** cancela o timer anterior sempre que `valor` muda.
- Efeitos não são para cálculos: estado derivado se calcula na renderização.

Consumindo a API no navegador

hooks/useProdutos.js

```
export function useProdutos({ nome = '', page = 0, size = 6 }) {
  const consulta = new URLSearchParams({ nome, page, size }).toString();
  const [resultado, setResultado] = useState({ consulta: null, dados: null, erro: null });

  useEffect(() => {
    const controle = new AbortController();

    fetch(`${API_URL}/api/produtos?${consulta}`, { signal: controle.signal })
      .then((resposta) => {
        if (!resposta.ok) throw new Error(`Erro ${resposta.status}`);
        return resposta.json();
      })
      .then((dados) => setResultado({ consulta, dados, erro: null }))
      .catch((e) => {
        if (e.name !== 'AbortError') setResultado({ consulta, dados: null, erro: e.message });
      });

    return () => controle.abort();  // cancela a requisição anterior
  }, [consulta]);

  return {
    dados: resultado.dados,
    erro: resultado.erro,
    carregando: resultado.consulta !== consulta,
  };
}
```

- `fetch` dentro de um **efeito**: roda no navegador, depois da renderização.
- Trata os três estados de uma requisição: **carregando**, **dados** e **erro**.
- **AbortController** cancela a requisição anterior: respostas antigas não sobrescrevem as novas.
- `carregando` é **derivado**: a consulta atual ainda não tem resultado.

Hooks personalizados

app/explorar/page.js

```
export default function ExplorarPage() {
  const [nome, setNome] = useState('');
  const [page, setPage] = useState(0);
  const nomeAtrasado = useDebounce(nome);
  const { dados, carregando, erro } = useProdutos({ nome: nomeAtrasado, page });
  const campoBusca = useRef(null);

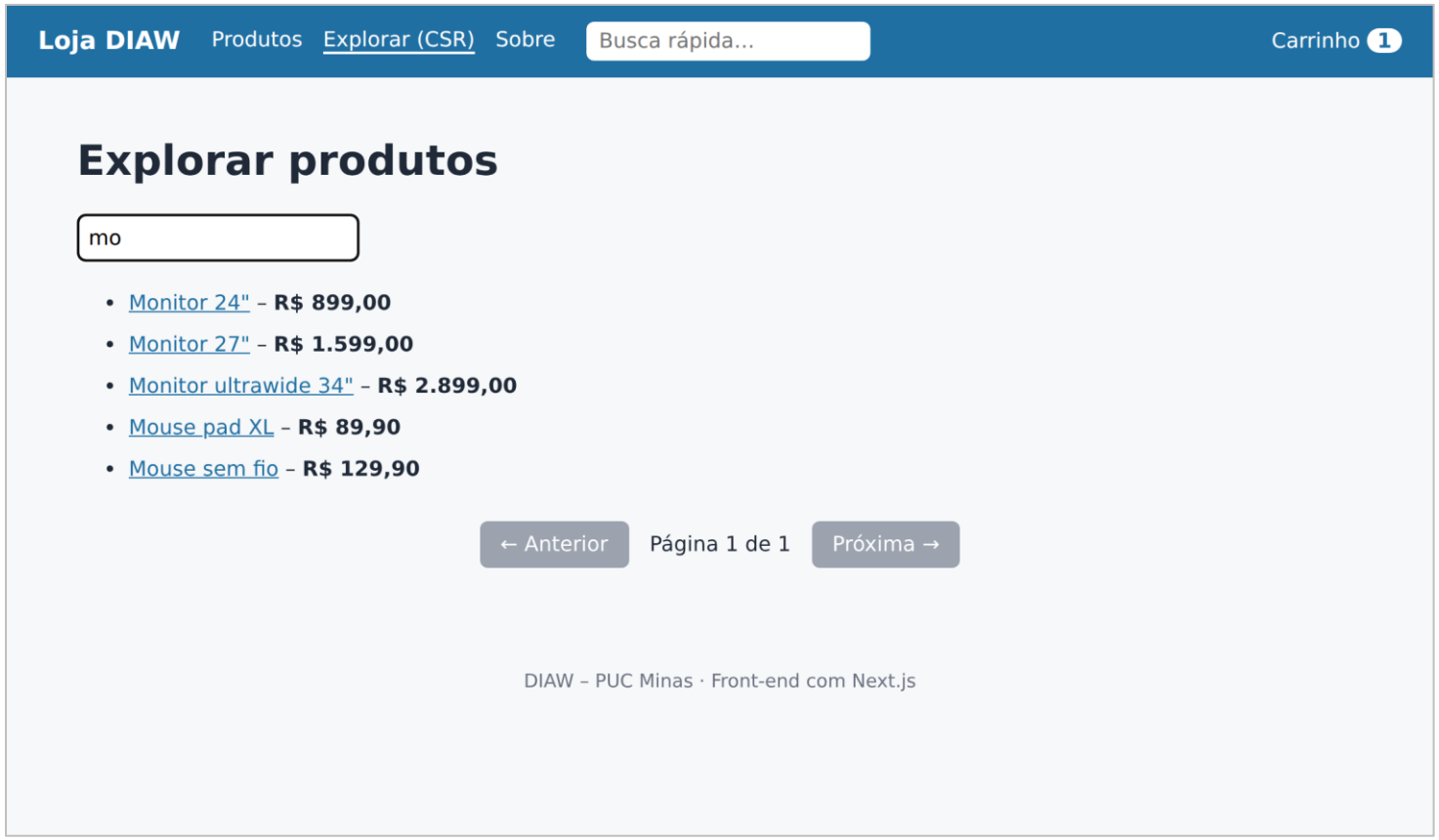
  useEffect(() => campoBusca.current.focus(), []);  // foco ao abrir a página
```

- Um **hook personalizado** é uma função que começa com **use** e usa outros hooks.
- Reaproveita **lógica com estado**, não interface.
- A página fica legível: “busque produtos com o nome atrasado”.

Regras dos hooks

1. Chame hooks **só no nível superior** do componente – nunca dentro de if, laços ou funções internas. 2. Chame hooks **só em componentes ou em outros hooks**. O plugin do ESLint aponta as violações.

HANDS ON – A página Explorar (CSR)



Digite no campo e observe a aba **Rede** das ferramentas do navegador:

- as requisições saem **do navegador** para a API (porta 8080 ou 3000);
- o **debounce** evita uma requisição por tecla;
- requisições canceladas aparecem como *canceled*.

Pare a API e digite de novo: a mensagem de erro vem do estado `erro`.

useRef: referência que não renderiza

components/BuscaRapida.js

```
const campo = useRef(null);
// …
function limpar() {
  setTermo('');
  campo.current.blur();  // useRef: acesso direto ao elemento
}
    <input ref={campo} value={termo} placeholder="Busca rápida…"
           onChange={(e) => setTermo(e.target.value)} />
```

app/explorar/page.js

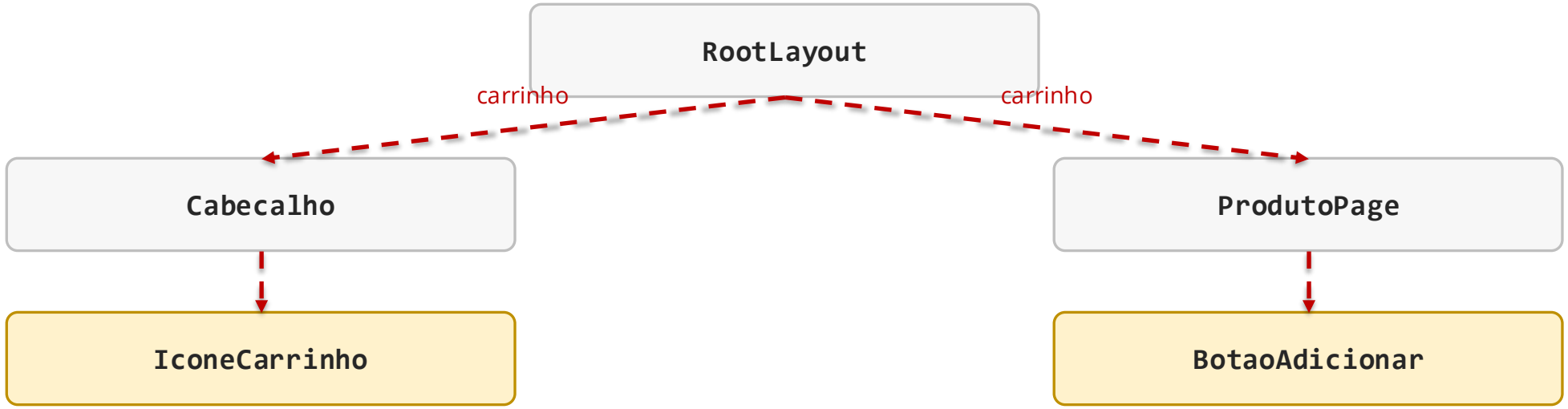
```
const campoBusca = useRef(null);

useEffect(() => campoBusca.current.focus(), []);  // foco ao abrir a página
```

- `useRef` guarda um valor em `.current` que **sobrevive** às renderizações, mas **não** provoca nova renderização ao mudar.
- Uso mais comum: acessar um **elemento do DOM** – foco, rolagem, medir tamanho.
- Outros usos: guardar o id de um timer ou um valor anterior.
- Se o valor aparece na tela, é **estado**, não ref.

O problema: prop drilling

O carrinho é usado no **cabeçalho** (contador), na **página de detalhe** (botão) e na **página do carrinho**. Passar por props obrigaria todos os intermediários a repassar dados que não usam.



Solução: Context API
Um **Provider** no topo da árvore oferece o valor; qualquer descendente o lê com `useContext`, sem intermediários.

Context API

contexts/CarrinhoContext.js (1/2)

```
const CarrinhoContext = createContext(null);

export function CarrinhoProvider({ children }) {
  const [estado, dispatch] = useReducer(carrinhoReducer, inicial);
  // … efeitos do localStorage

  const { itens } = estado;
  const total = itens.reduce((soma, i) => soma + i.preco * i.qtd, 0);
  const quantidade = itens.reduce((soma, i) => soma + i.qtd, 0);

  return (
    <CarrinhoContext.Provider value={{ itens, total, quantidade, dispatch }}>
      {children}
    </CarrinhoContext.Provider>
  );
}
```

(2/2)

```
export function useCarrinho() {
  const contexto = useContext(CarrinhoContext);
  if (!contexto) {
    throw new Error('useCarrinho fora do CarrinhoProvider');
  }
  return contexto;
}
```

IconeCarrinho.js

```
const { quantidade } = useCarrinho();
```

O Provider envolve toda a aplicação no **layout raiz**.

useReducer: estado com regras

contexts/CarrinhoContext.js

```
function carrinhoReducer(estado, acao) {
  const { itens } = estado;
  switch (acao.tipo) {
    case 'carregar':
      return { itens: acao.itens, carregado: true };
    case 'adicionar': {
      const existe = itens.some((i) => i.id === acao.produto.id);
      const { id, nome, preco } = acao.produto;
      return {
        ...estado,
        itens: existe
          ? itens.map((i) => (i.id === id ? { ...i, qtd: i.qtd + 1 } : i))
          : [...itens, { id, nome, preco, qtd: 1 }],
      };
    }
    case 'remover':
      return { ...estado, itens: itens.filter((i) => i.id !== acao.id) };
    case 'limpar':
      return { ...estado, itens: [] };
    default:
      throw new Error(`Ação desconhecida: ${acao.tipo}`);
  }
}
```

- O **reducer** é uma função **pura**: `(estado, ação) → novo estado`.
- Os componentes só **despacham ações**; as regras ficam num lugar só.
- Fácil de testar sem React.

nos componentes

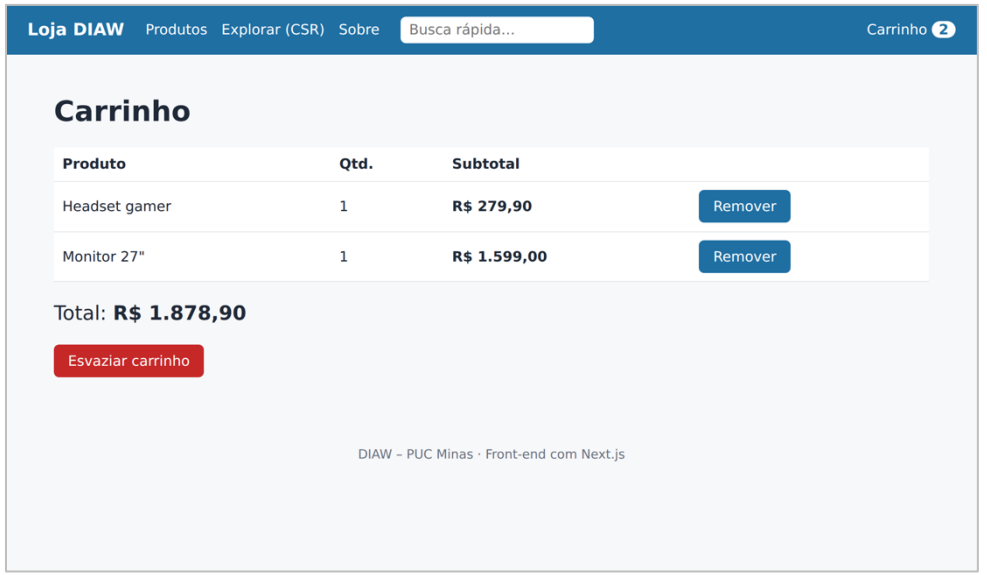
```
dispatch({ tipo: 'adicionar', produto });
dispatch({ tipo: 'remover', id });
dispatch({ tipo: 'limpar' });
```

Persistindo o carrinho no localStorage

contexts/CarrinhoContext.js

```
// localStorage só existe no navegador: lê depois da montagem
useEffect(() => {
  const salvo = JSON.parse(localStorage.getItem('carrinho') ?? '[]');
  dispatch({ tipo: 'carregar', itens: salvo });
}, []);

// Persiste a cada alteração (depois de carregar)
useEffect(() => {
  if (estado.carregado) {
    localStorage.setItem('carrinho', JSON.stringify(estado.itens));
  }
}, [estado]);
```



- `localStorage` **só existe no navegador**: por isso é lido num efeito, depois da montagem – nunca durante a renderização.
- O flag `carregado` impede que o estado inicial vazio sobrescreva o carrinho salvo antes da leitura.
- Em desenvolvimento, o **StrictMode** executa os efeitos duas vezes de propósito, para revelar efeitos sem limpeza.

Qual hook usar?

Hook	Use quando...	Na loja-web
useState	um valor muda e aparece na tela	BotaoAdicionar, explorar
useEffect	é preciso sincronizar com algo externo (rede, timer, storage)	useProdutos, useDebounce
useRef	um valor ou elemento não deve provocar renderização	foco do campo de busca
useContext	vários componentes distantes leem o mesmo dado	useCarrinho
useReducer	o estado tem várias ações e regras de transição	carrinhoReducer
useTransition	uma atualização pode demorar e a tela deve continuar responsiva	Filtros, BotaoExcluir
useActionState	um formulário chama uma Server Action e exibe o resultado	FormularioProduto

Os dois últimos dependem do Next.js e aparecem na parte de busca de dados.

PUC Minas

React e Next.js

Next.js e o App Router

Prof. Rommel Vieira Carneiro

Roteamento baseado em pastas

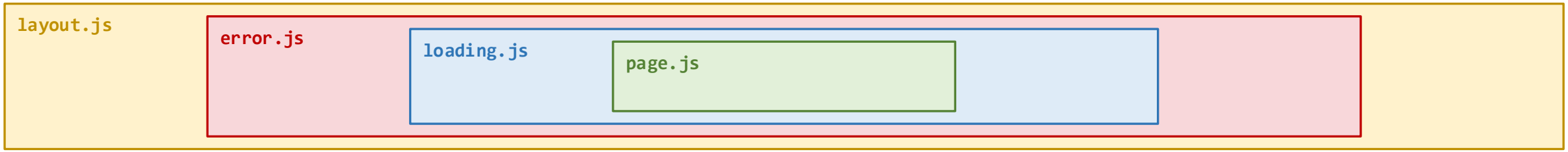
```
app/
├── page.js                    /
├── sobre/page.js              /sobre
├── explorar/page.js           /explorar
├── carrinho/page.js           /carrinho
├── produtos/
│   ├── page.js                /produtos
│   ├── novo/page.js           /produtos/novo
│   └── [id]/
│       ├── page.js            /produtos/64a1…
│       └── editar/page.js     /produtos/64a1…/editar
└── api/sugestoes/route.js     /api/sugestoes (JSON)
```

- Cada **pasta** é um segmento da URL.
- A rota só existe se a pasta tiver um **page.js** (ou **route.js**).
- **[id]** entre colchetes é um segmento **dinâmico**.
- Outros arquivos na pasta (componentes, estilos) **não** viram rota.

Compare: no Express as rotas são declaradas em código (router.get); no Next.js, a estrutura de pastas é a tabela de rotas.

Arquivos especiais de uma rota

Arquivo	Papel	Na loja-web
page.js	Conteúdo da rota – torna a URL acessível	todas as páginas
layout.js	Moldura compartilhada; preserva estado ao navegar	app/layout.js
loading.js	Tela exibida enquanto a página carrega (Suspense)	produtos/loading.js
error.js	Tela de erro da rota (error boundary) – Client Component	produtos/error.js
not-found.js	Tela para notFound() ou URL inexistente	produtos/[id]/not-found.js
route.js	Endpoint HTTP (GET, POST...) em vez de página	api/sugestoes/route.js



Layouts

app/layout.js

```
import './globals.css';
import Cabecalho from '@/components/Cabecalho';
import { CarrinhoProvider } from '@/contexts/CarrinhoContext';

export const metadata = {
  title: { default: 'Loja DIAW', template: '%s | Loja DIAW' },
  description: 'Front-end da loja da disciplina DIAW com Next.js',
};

// Layout raiz: envolve todas as páginas da aplicação
export default function RootLayout({ children }) {
  return (
    <html lang="pt-BR">
      <body>
        <CarrinhoProvider>
          <Cabecalho />
          <main>{children}</main>
          <footer>DIAW – PUC Minas · Front-end com Next.js</footer>
        </CarrinhoProvider>
      </body>
    </html>
  );
}
```

- O **layout raiz** é obrigatório e define `<html>` e `<body>`.
- `children` é a página da rota atual.
- Ao navegar, o layout **não é recriado**: cabeçalho e carrinho mantêm o estado.
- Pastas podem ter layouts próprios, **aninhados** dentro do raiz.
- `metadata` define `<title>` e descrição; `template` compõe títulos como “Produtos | Loja DIAW”.

Rotas dinâmicas

app/produtos/[id]/page.js

```
export async function generateMetadata({ params }) {
  const produto = await buscarProduto((await params).id);
  return { title: produto?.nome ?? 'Produto não encontrado' };
}

// Rota dinâmica: /produtos/[id]
export default async function ProdutoPage({ params }) {
  const { id } = await params;
  const produto = await buscarProduto(id);
  if (!produto) notFound();

  return (
    <article className="detalhe">
      <span className="etiqueta">{produto.categoria}</span>
      <h1>{produto.nome}</h1>
      {produto.descricao && <p>{produto.descricao}</p>}
      <p className="grande"><Preco valor={produto.preco} /></p>
      <p>{produto.disponivel ? `${produto.estoque} em estoque` : 'Esgotado'}</p>
      {/* … ações e relacionados */}
    </article>
  );
}
```

- A pasta **[id]** captura o segmento: `/produtos/64a1` → `{ id: '64a1' }`.
- No Next.js 15+, **params é uma Promise**: use `await params`.
- `notFound()` interrompe a renderização e exibe o `not-found.js` mais próximo.
- `generateMetadata` gera o título com o nome do produto.

Query string com searchParams

app/produtos/page.js

```
// Server Component assíncrono: busca os dados no servidor (SSR)
export default async function ProdutosPage({ searchParams }) {
  const { nome = '', categoria = '', page = '0' } = await searchParams;

  const [pagina, categorias] = await Promise.all([
    listarProdutos({ nome, categoria, page }),
    listarCategorias(),
  ]);

  return (
    <section>
      <div className="titulo">
        <h1>Produtos</h1>
        <Link href="/produtos/novo" className="botao">+ Novo produto</Link>
      </div>
      <Filtros categorias={categorias} />
      <p className="total">{pagina.page.totalElements} produto(s)</p>
      <GradeProdutos produtos={pagina.content} />
      <Paginacao pagina={pagina.page} filtros={{ nome, categoria }} />
    </section>
  );
}
```

- `/produtos?nome=mouse&page=1` → `searchParams` (também uma Promise).
- Valores chegam como **texto**; ausentes ficam `undefined` – daí os valores padrão.
- `Promise.all` busca produtos e categorias **em paralelo**.
- Usar searchParams torna a página **dinâmica**: renderizada a cada requisição.

Navegação: Link e useRouter

declarativa

```
<Link href={`/produtos/${produto.id}`}>{produto.nome}</Link>
```

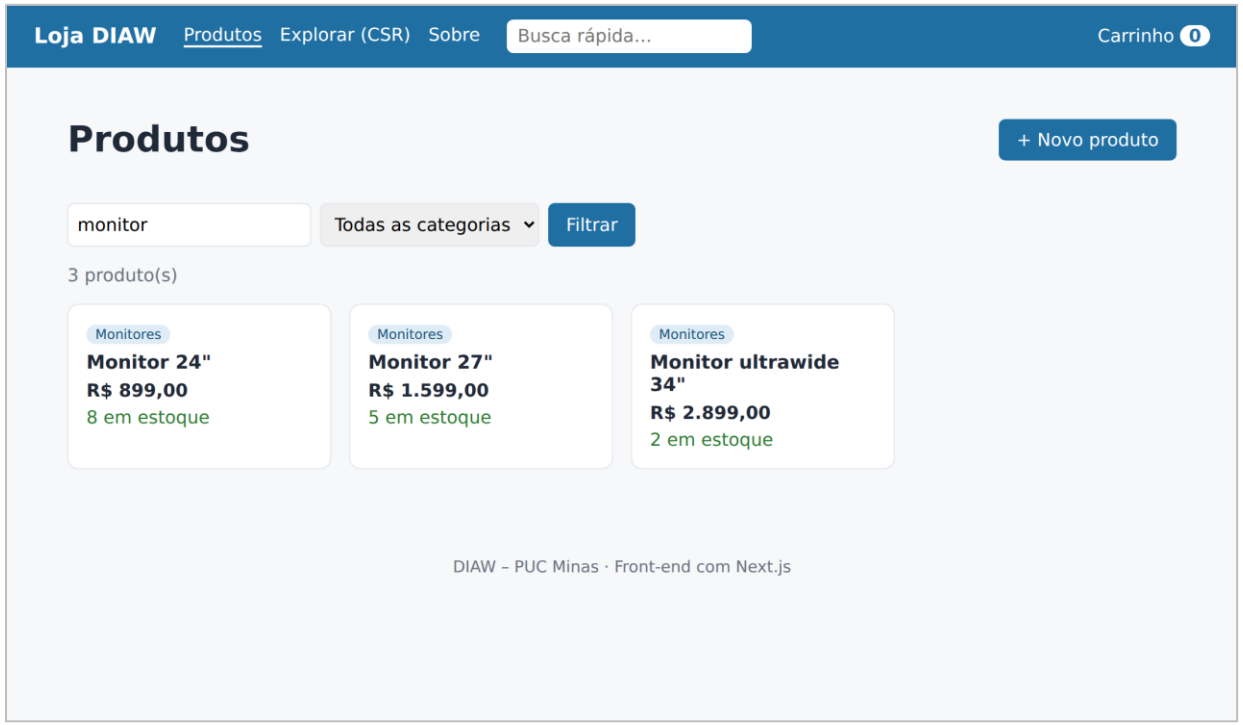
programática – components/Filtros.js

```
const router = useRouter();
const searchParams = useSearchParams();
const [pendente, startTransition] = useTransition();

function aplicar(evento) {
  evento.preventDefault();
  const dados = new FormData(evento.currentTarget);
  const params = new URLSearchParams();
  for (const [campo, valor] of dados) if (valor) params.set(campo, valor);
  startTransition(() => router.push(`/produtos?${params}`));
}
```

- **<Link>** navega sem recarregar a página e **pré-carrega** as rotas visíveis na tela.
  - Use `<a>` só para endereços externos.
- **useRouter** navega por código: após um evento, um filtro, um formulário.
  - `router.push(url)` · `router.back()` · `router.refresh()`
- Outros hooks de navegação:
  - `usePathname()` – caminho atual (NavLink)
  - `useSearchParams()` – lê a query string no cliente
- Os hooks de navegação só funcionam em **Client Components**.

A URL como estado: filtros e paginação



components/Paginacao.js

```
export default function Paginacao({ pagina, filtros }) {
  const { number, totalPages } = pagina;
  if (totalPages <= 1) return null;

  const href = (page) => {
    const params = new URLSearchParams({ page });
    for (const [campo, valor] of Object.entries(filtros)) {
      if (valor) params.set(campo, valor);
    }
    return `/produtos?${params}`;
  };
```

- Filtro e página ficam **na URL**, não em useState: o link pode ser **compartilhado**, favoritado e o botão **Voltar** funciona.
- `Filtros` só muda a URL; a **página no servidor** lê `searchParams` e busca os dados.

PUC Minas

React e Next.js

Estratégias de Renderização

Prof. Rommel Vieira Carneiro

Onde e quando o HTML é gerado?

CSR **Client-Side Rendering**

O servidor envia um HTML quase vazio + JavaScript; o **navegador** busca os dados e monta a tela.

`Na loja-web: /explorar, /carrinho`

SSR **Server-Side Rendering**

O **servidor** gera o HTML com os dados **a cada requisição**.

`Na loja-web: /produtos, /produtos/[id]`

SSG **Static Site Generation**

O HTML é gerado **uma vez, no build**, e servido pronto a todos.

`Na loja-web: /, /sobre`

ISR **Regeneração incremental**

HTML estático que é **regenerado** depois de um intervalo ou sob demanda.

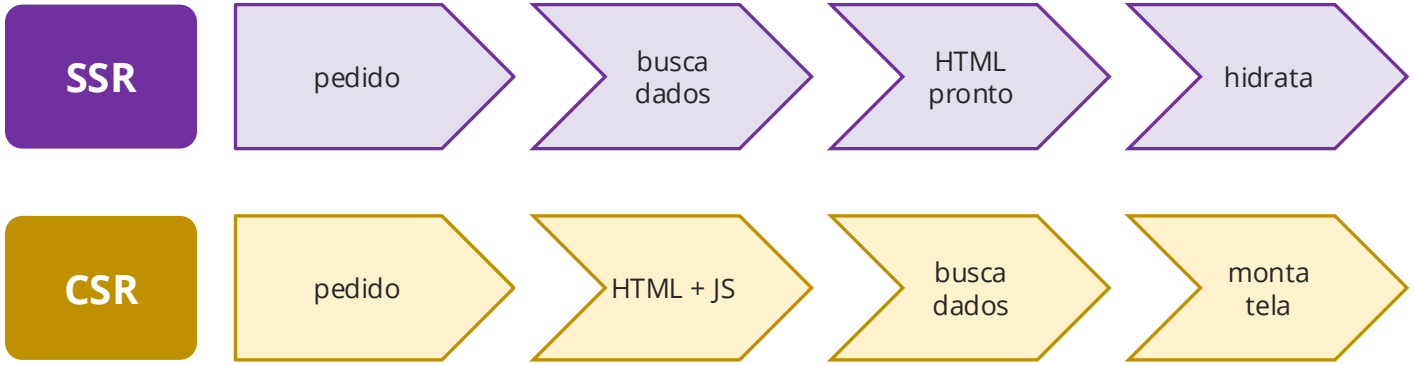
`Na loja-web: /produtos/novo`

CSR × SSR na prática

terminal

```
$ curl -s localhost:3001/produtos | grep -o "Headset gamer"
Headset gamer

$ curl -s localhost:3001/explorar | grep -o "Headset gamer"
(nenhum resultado: a lista é montada no navegador)
```



- **SSR**: o conteúdo já está no HTML – bom para **SEO**, links compartilhados e primeira exibição rápida.
- **CSR**: o HTML chega vazio; buscadores e pré-visualizações de links podem não ver o conteúdo.
- **Hidratação**: o React "liga" os eventos ao HTML que veio do servidor.
- CSR ainda é útil em telas privadas e muito interativas, onde SEO não importa.

O build revela a estratégia de cada rota

terminal

```
$ npm run build

Route (app)                 Revalidate  Expire
┌ ○ /
├ ○ /_not-found
├ ƒ /api/sugestoes
├ ○ /carrinho
├ ○ /explorar
├ ƒ /produtos
├ ƒ /produtos/[id]
├ ƒ /produtos/[id]/editar
├ ○ /produtos/novo                  1h      1y
└ ○ /sobre


○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

- ○ **Static**: pré-renderizada no build (SSG). /explorar e /carrinho também: o HTML é estático e os dados vêm no navegador.
- ƒ **Dynamic**: renderizada a cada requisição (SSR) – usa `searchParams`, `params` ou dados sem cache.
- **Revalidate 1h**: /produtos/novo é estática, mas é regenerada quando o cache de categorias (3600 s) expira – ISR.
- O Next.js **escolhe sozinho** a partir do que a rota usa.

Server Components e Client Components

	Server Component (padrão)	Client Component ('use client')
Executa em	Servidor (build ou requisição)	Servidor (HTML inicial) e navegador
Pode ser async / buscar dados	Sim – await direto no componente	Não – usa efeitos ou recebe props
Hooks (useState, useEffect...)	Não	Sim
Eventos (onClick, onChange)	Não	Sim
APIs do navegador (localStorage)	Não	Sim, em efeitos e eventos
Segredos e variáveis de ambiente	Sim	Só NEXT_PUBLIC_*
JavaScript enviado ao navegador	Nenhum	O código do componente

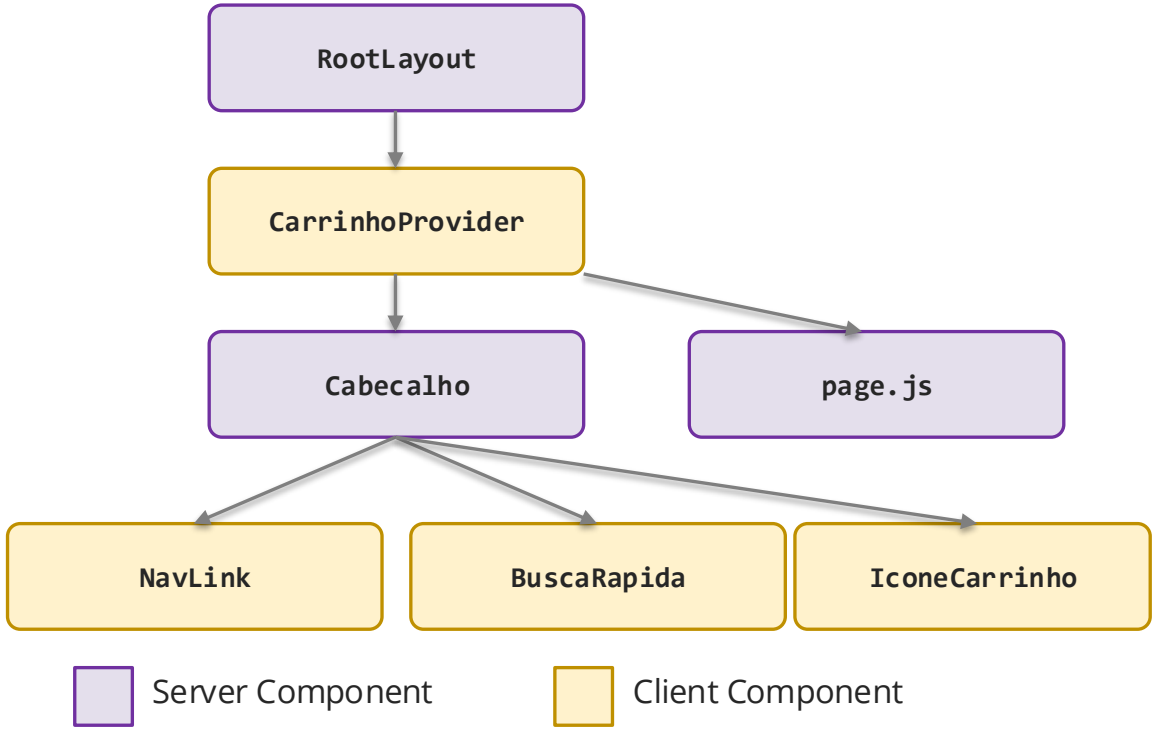
Regra prática: comece com Server Components e marque 'use client' só nas “folhas” que precisam de interatividade.

A fronteira 'use client'

components/Cabecalho.js (servidor)

```
import Link from 'next/link';
import NavLink from './NavLink';
import BuscaRapida from './BuscaRapida';
import IconeCarrinho from './IconeCarrinho';

// Server Component que compõe Client Components interativos
export default function Cabecalho() {
  return (
    <header className="cabecalho">
      <Link href="/" className="marca">Loja DIAW</Link>
      <nav>
        <NavLink href="/produtos">Produtos</NavLink>
        <NavLink href="/explorar">Explorar (CSR)</NavLink>
        <NavLink href="/sobre">Sobre</NavLink>
      </nav>
      <BuscaRapida />
      <IconeCarrinho />
    </header>
  );
}
```



- `'use client'` no topo do arquivo marca a **fronteira**: esse componente e o que ele importa vão para o navegador.
- Um Client Component pode receber Server Components como **children** – por isso `CarrinhoProvider` envolve páginas que continuam no servidor.

Restrições e cuidados

Props serializáveis

Do servidor para o cliente só passam dados (texto, número, objetos, arrays, datas). **Funções não passam** – exceto Server Actions.

server-only

`import 'server-only'` em lib/api.js faz o **build falhar** se um Client Component importar o arquivo por engano.

NEXT_PUBLIC_

Variáveis com esse prefixo são **copiadas no JavaScript no build** e ficam públicas. Nunca coloque segredos nelas.

Sem window no servidor

`localStorage`, `window` e `document` só em efeitos e eventos – o componente cliente também renderiza no servidor.

Suspense e streaming

app/produtos/[id]/page.js

```
<Suspense fallback={<p className="carregando">Carregando relacionados…</p>}>
  <Relacionados categoriaId={produto.categoriaId} atual={id} />
</Suspense>
```

Relacionados.js

```
export default async function Relacionados({ categoriaId, atual }) {
  const { content } = await listarProdutos({ categoria: categoriaId, size: 5 });
  const outros = content.filter((p) => p.id !== atual).slice(0, 4);
```

Sem bloquear a página
O detalhe do produto aparece antes; só a seção de relacionados espera a segunda consulta à API.

cabeçalho + detalhe (HTML imediato)

"Carregando relacionados…" (fallback)

relacionados chegam depois, no mesmo fluxo

- `<Suspense>` mostra um **fallback** enquanto um componente assíncrono espera dados.
- O servidor envia o HTML **em partes** (streaming): o usuário vê a página antes de tudo estar pronto.
- `loading.js` é um Suspense automático em volta da página inteira.

loading.js, error.js e not-found.js

app/produtos/loading.js

```
// Exibido imediatamente enquanto a página busca os dados (streaming)
export default function Carregando() {
  return (
    <section>
      <h1>Produtos</h1>
      <div className="grade">
        {Array.from({ length: 8 }, (_, i) => (
          <div key={i} className="cartao esqueleto" />
        ))}
      </div>
    </section>
  );
}
```

app/produtos/error.js

```
'use client'; // limites de erro precisam ser Client Components

export default function ErroProdutos({ error, retry }) {
  return (
    <section className="alerta">
      <h2>Não foi possível carregar os produtos</h2>
      <p>Verifique se a API está em execução.</p>
      <button onClick={() => retry()}>Tentar novamente</button>
    </section>
  );
}
```

- **loading.js**: esqueleto exibido na hora, enquanto a página busca dados – a navegação parece instantânea.
- **error.js**: captura exceções da rota (API fora do ar); precisa ser Client Component. `retry()` tenta renderizar de novo.
- **not-found.js**: exibido por `notFound()`; o mais próximo na árvore de pastas vence.

Escolhendo a estratégia

Situação	Estratégia	Exemplo
Conteúdo igual para todos, muda raramente	SSG	Home, Sobre, documentação
Muda de tempos em tempos; pode ficar alguns segundos desatualizado	SSG/SSR + revalidate (ISR)	Catálogo, blog, categorias
Depende da requisição (filtros, usuário, cookies)	SSR (dinâmica)	Busca, lista filtrada
Interação intensa, privada, sem SEO	CSR em Client Component	Carrinho, painel, editor
Parte lenta numa página rápida	Suspense + streaming	Relacionados, recomendações

Não é tudo ou nada
A mesma página mistura estratégias: /produtos/[id] é SSR, o botão Adicionar é interativo no cliente e os relacionados chegam por streaming.

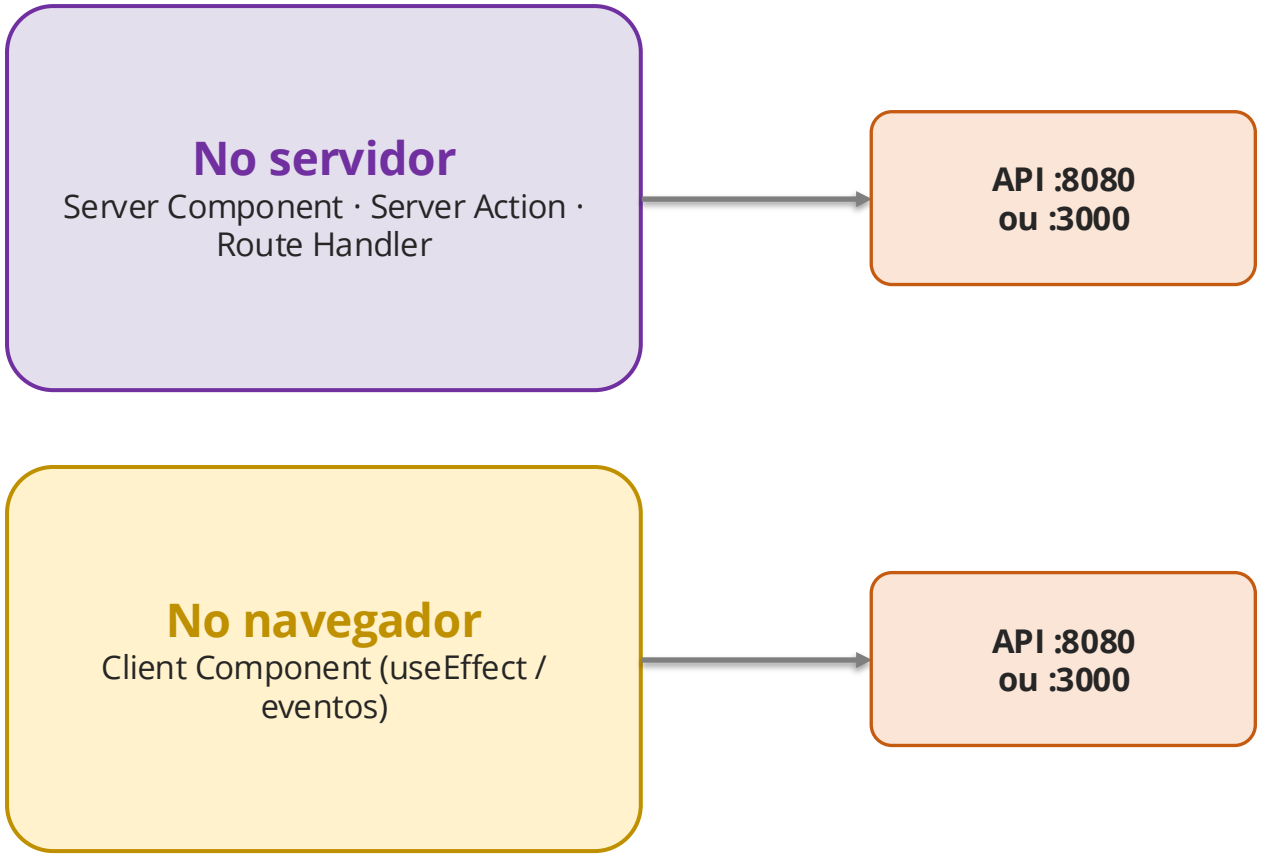
PUC Minas

React e Next.js

Busca de Dados e Atualizações

Prof. Rommel Vieira Carneiro

Dois caminhos até a API



- variável API_URL (privada)
- sem CORS: é servidor → servidor
- pode usar o cache do Next.js
- segredos e tokens ficam protegidos

- variável NEXT_PUBLIC_API_URL (pública)
- a API precisa liberar CORS
- sem cache do Next.js
- a API fica exposta ao usuário

Na loja-web, só a página /explorar busca no navegador – de propósito, para comparar.

Acesso à API no servidor

lib/api.js

```
// Acesso à API da loja – executado SOMENTE no servidor Next.js
import 'server-only';

const API_URL = process.env.API_URL;

async function obter(caminho, opcoes) {
  const resposta = await fetch(`${API_URL}${caminho}`, opcoes);
  if (resposta.status === 404) return null;
  if (!resposta.ok) {
    throw new Error(`Falha ao consultar a API (${resposta.status})`);
  }
  return resposta.json();
}

export function listarProdutos({ nome, categoria, page = 0, size = 8 } = {}) {
  const params = new URLSearchParams({ page, size });
  if (nome) params.set('nome', nome);
  if (categoria) params.set('categoria', categoria);
  return obter(`/api/produtos?${params}`, {
    next: { revalidate: 30, tags: ['produtos'] },
  });
}

export function buscarProduto(id) {
  return obter(`/api/produtos/${encodeURIComponent(id)}`, {
    next: { revalidate: 30, tags: ['produtos'] },
  });
}
```

- Um único módulo concentra o acesso à API – como o **repository** das unidades anteriores.
- `obter` traduz o HTTP: **404** → **null**, outros erros → exceção (capturada pelo `error.js`).
- As páginas chamam funções com nomes do domínio: `listarProdutos`, `buscarProduto`.
- A opção `next` controla o **cache** (próximo slide).

CORS: quando o navegador chama a API

Navegador em localhost:3001
fetch('http://localhost:8080/api/produtos')

API responde com o cabeçalho
Access-Control-Allow-Origin: http://localhost:3001

Navegador libera a resposta
origem autorizada → o JavaScript lê o JSON

Express

```
// loja-mvc – routes/api.routes.js
router.use(cors({ origin: 'http://localhost:3001',
                  exposedHeaders: ['Location'] }));
```

Spring Boot

```
// loja-api – config/CorsConfig.java
registry.addMapping("/api/**")
    .allowedOrigins(origens)   // app.cors.origens
    .exposedHeaders("Location");
```

- **Origem** = protocolo + domínio + porta. 3001 → 8080 é outra origem: sem CORS, o navegador **bloqueia** a leitura da resposta.
- CORS é regra **do navegador**: chamadas feitas pelo servidor Next.js não passam por ela. Nunca use `origin: '*'` em APIs com login.

Cache e revalidação

lib/api.js

```
return obter(`/api/produtos?${params}`, {
    next: { revalidate: 30, tags: ['produtos'] },
  });
```

Opção	Efeito
next: { revalidate: 30 }	reaproveita a resposta por 30 s
next: { tags: ['produtos'] }	rotula para invalidar depois
cache: 'no-store'	sempre busca na API

O Next.js 16 traz um modelo opcional, **Cache Components** (`'use cache'`), que torna o cache explícito por componente. Usamos o modelo com `fetch`, ainda o padrão.

0–30 s
resposta do cache (rápido)

após 30 s
entrega o cache e atualiza em segundo plano

updateTag('produtos')
descarta o cache na hora

Atenção: dados desatualizados
Alterações feitas **direto na API** (curl, outro sistema) só aparecem após o tempo de revalidação. Em desenvolvimento, isso confunde: lembre do cache.

Formulários com Server Actions

lib/actions.js

```
'use server';

import { updateTag } from 'next/cache';
import { redirect } from 'next/navigation';
const API_URL = process.env.API_URL;

export async function salvarProduto(estadoAnterior, formData) {
  const id = formData.get('id');
  const produto = {
    nome: formData.get('nome'),
    descricao: formData.get('descricao') || null,
    preco: Number(formData.get('preco')),
    estoque: Number(formData.get('estoque')),
    categoriaId: formData.get('categoriaId'),
  };
  const resposta = await fetch(`${API_URL}/api/produtos${id ? `/${id}` : ''}`, {
    method: id ? 'PUT' : 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(produto),
  });

  if (!resposta.ok) {
    const problema = await resposta.json();
    return { mensagem: problema.detail, erros: problema.erros ?? {}, valores: produto };
  }
  const salvo = await resposta.json();
  updateTag('produtos');  // invalida o cache das listagens
  redirect(`/produtos/${salvo.id}`);
}
```

- `'use server'` marca funções que rodam **no servidor**, mas podem ser chamadas por formulários do cliente.
- Recebem um **FormData** com os campos pelo atributo `name`.
- Em erro 400, devolvem o Problem Details para a tela.
- Em sucesso: `updateTag` invalida o cache e `redirect` leva ao detalhe.

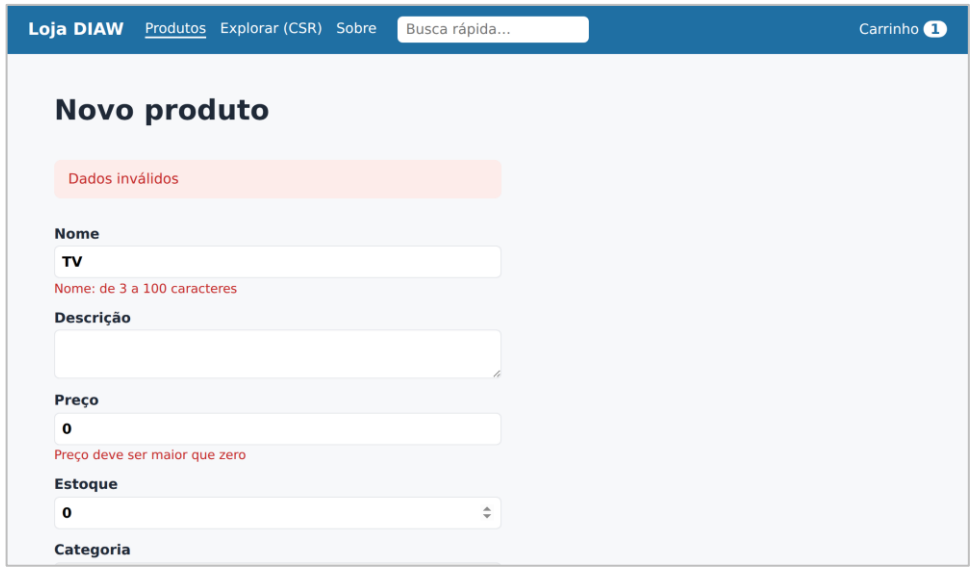
useActionState: estado do formulário

components/FormularioProduto.js

```
export default function FormularioProduto({ produto = {}, categorias }) {
  const [estado, acao, pendente] = useActionState(salvarProduto, {});
  const valores = estado.valores ?? produto;
  const erro = (campo) =>
    estado.erros?.[campo] && <span className="erro">{estado.erros[campo]}</span>;

  return (
    <form action={acao} className="formulario">
      {estado.mensagem && <p className="alerta">{estado.mensagem}</p>}
      <input type="hidden" name="id" value={produto.id ?? ''} />

      <label>Nome
        <input name="nome" defaultValue={valores.nome} />{erro('nome')}
      </label>      {/* … demais campos */}
      <button type="submit" disabled={pendente}>…</button>
    </form>
  );
}
```



- `useActionState(acao, inicial)` devolve **[estado, ação para o form, pendente]**. O retorno da Server Action vira o novo `estado`.
- Os erros por campo vêm da **API** (Spring ou Express): uma única fonte de validação. `valores` mantém o que o usuário digitou.

Atualizando a interface após a escrita

components/BotaoExcluir.js

```
import { useTransition } from 'react';
import { excluirProduto } from '@/lib/actions';

// Chama uma Server Action a partir de um evento do navegador
export default function BotaoExcluir({ id, nome }) {
  const [pendente, startTransition] = useTransition();

  function excluir() {
    if (!confirm(`Excluir "${nome}"?`)) return;
    startTransition(() => excluirProduto(id));
  }

  return (
    <button className="perigo" onClick={excluir} disabled={pendente}>
      {pendente ? 'Excluindo…' : 'Excluir'}
    </button>
  );
}
```

1. Clique
confirm + startTransition

2. Server Action
DELETE na API

3. updateTag('produtos')
cache da listagem descartado

4. redirect('/produtos')
lista já sem o produto

- Server Actions também podem ser chamadas **fora de formulários**, como uma função assíncrona. `useTransition` informa que a ação está pendente.
- Sem `updateTag`, a lista continuaria mostrando o produto excluído até o cache expirar.

Route Handlers: endpoints no Next.js

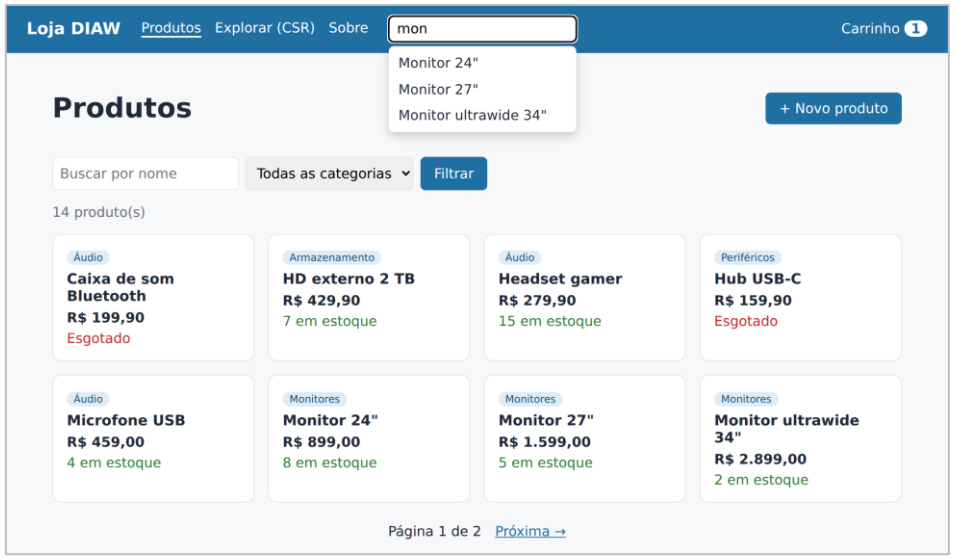
app/api/sugestoes/route.js

```
// Route Handler: endpoint HTTP do próprio Next.js (GET /api/sugestoes?q=...)
const API_URL = process.env.API_URL;

export async function GET(request) {
  const q = request.nextUrl.searchParams.get('q')?.trim() ?? '';
  if (q.length < 2) return Response.json([]);

  const params = new URLSearchParams({ nome: q, size: 5 });
  const resposta = await fetch(`${API_URL}/api/produtos?${params}`);
  if (!resposta.ok) {
    return Response.json({ erro: 'API indisponível' }, { status: 502 });
  }

  const { content } = await resposta.json();
  return Response.json(content.map(({ id, nome }) => ({ id, nome })));
}
```



- Um **route.js** exporta funções com o nome do método HTTP (GET, POST…) e devolve um `Response` – como um controller.
- A busca rápida chama `/api/sugestoes` **na mesma origem**: sem CORS, e a resposta só leva `id` e `nome` – um **BFF** (Backend for Frontend).

Server Action, Route Handler ou fetch?

	Server Action	Route Handler	fetch no Client Component
Para quê	Escrita disparada pela própria UI	Endpoint HTTP com URL própria	Leitura interativa no navegador
Quem chama	Formulários e eventos do app	Qualquer cliente: app, webhook, outro sistema	O navegador, direto na API
Método	Sempre POST (automático)	GET, POST, PUT, DELETE…	Qualquer
CORS	Não se aplica	Mesma origem do app	A API precisa liberar
Na loja-web	salvarProduto, excluirProduto	/api/sugestoes	useProdutos (/explorar)

Regra prática
Leitura → Server Component. Escrita a partir da interface → Server Action. Endpoint para terceiros ou para o navegador consultar → Route Handler.

HANDS ON – Trocando de back-end

terminal

```
# Terminal 1 – escolha UMA das APIs
$ cd loja-api && ./mvnw spring-boot:run     # :8080
$ cd loja-mvc && npm start                  # :3000

# Terminal 2 – front-end
$ cd loja-web
$ cp .env.example .env.local
#   API_URL=http://localhost:8080
#   NEXT_PUBLIC_API_URL=http://localhost:8080
$ npm install
$ npm run dev                               # :3001
```

1. Suba a API Spring e navegue: listar, filtrar, paginar, cadastrar com erro, editar e excluir

2. Troque as duas variáveis para a porta 3000 e reinicie o `npm run dev`

3. Repita o roteiro com a API Express: o comportamento deve ser idêntico

4. Desafio rápido: qual variável muda sem reiniciar? Por que NEXT_PUBLIC_ exige novo build em produção?

Desafios

1 Ordenação na listagem

Adicione um select “ordenar por” (nome, preço) em `Filtros`, mantendo o valor na URL e repassando-o à API.

2 Página de categorias

Crie `/categorias/[id]` como Server Component, com `generateStaticParams` para pré-gerar uma página por categoria.

3 Quantidade no carrinho

Inclua ações `incrementar` e `decrementar` no `carrinhoReducer` e botões + / − na página do carrinho.

4 Busca no cabeçalho com Enter

Ao pressionar Enter na `BuscaRapida`, navegue para `/produtos?nome=…` com `useRouter`.

Boas práticas – síntese

Servidor

- Server Component por padrão
- Acesso à API em lib/ com server-only
- Server Actions para escrita + updateTag

Cliente

- 'use client' só onde há interação
- Estado mínimo; o resto é derivado
- Efeitos com limpeza (Abort, timers)

Rotas e dados

- Filtros e páginas na URL
- loading, error e not-found por rota
- Cache consciente: revalidate e tags

Transversais
Componentes pequenos e puros · keys estáveis · contrato de API documentado · segredos fora do NEXT_PUBLIC_ · CORS restrito às origens conhecidas · ESLint com as regras de hooks.

Referências

- BANKS, Alex; PORCELLO, Eve. **Learning React**: modern patterns for developing React apps. 2. ed. Sebastopol: O'Reilly, 2020.
- REACT. **React Documentation**. Disponível em: https://react.dev/learn
- REACT. **You Might Not Need an Effect**. Disponível em: https://react.dev/learn/you-might-not-need-an-effect
- VERCEL. **Next.js Documentation – App Router**. Disponível em: https://nextjs.org/docs/app
- VERCEL. **Next.js – Caching and Revalidating**. Disponível em: https://nextjs.org/docs/app/getting-started/caching-and-revalidating
- MDN WEB DOCS. **Cross-Origin Resource Sharing (CORS)**. Disponível em: https://developer.mozilla.org/docs/Web/HTTP/CORS
- NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457: Problem Details for HTTP APIs**. IETF, 2023.

PUC Minas

Obrigado!

Prof. Rommel Vieira Carneiro